

Theme Builder Guide

Theme Builder Guide

This guide explains how to build a Codeware theme. We'll show what files a theme is, what data its templates receive, and how to package & install it. Read it top to bottom the first time, then use the section headings to jump back in while you build.

How to build and install a theme for your Codeware site

1. What a theme is

A theme is a **single folder of Blade templates** that styles the public site. Every folder under `resources/views/frontend/themes/` is a theme; the folder name is the theme's **slug**. The Admin panel's Theme Settings page (`admin: theme-settings`) lists them automatically by scanning that directory, and whichever is marked **Live** in the picker is the active theme.

Themes are pure views — no routes, controllers, or models. Pages are created in the admin CMS and the framework renders them through the active theme's templates. A theme can be as small as the two required templates (home + page), or as large as the built-in **ecommerce** theme with shop, product, blog, cart and account pages.

2. Folder structure

```
resources/views/frontend/themes/
└── my-theme/
    ├── home.blade.php           # REQUIRED - homepage
    ├── page.blade.php           # REQUIRED - every standalone page
    ├── shop.blade.php           # optional - catalog (falls back to ecommerce)
    ├── product.blade.php        # optional - product detail
    ├── blog.blade.php           # optional - blog feed
    ├── post.blade.php           # optional - blog post detail
    ├── cart.blade.php           # optional - shopping cart
    ├── checkout.blade.php       # optional - checkout
    ├── favorites.blade.php       # optional - saved favorites
    ├── category.blade.php       # optional - category landing
    ├── brand.blade.php          # optional - brand landing
    ├── tag.blade.php            # optional - tag landing
    ├── order-confirmation.blade.php
    └── partials/                # your own partials (header, footer, cards...)
public storage copy lives at public/themes/my-theme/
├── style.css                   # linked via asset('themes/my-theme/style.css')
└── script.js                   # linked via asset('themes/my-theme/script.js')
```

The home and page templates are always rendered from the **active** theme. Every other storefront view (shop, product, category, brand, tag, blog, post, cart, checkout, favorites, order-confirmation, account dashboard, account orders, account order, account profile) uses `Themes::view('name')`: your theme's template when you ship one, otherwise the **ecommerce** theme's. That means a one-page portfolio theme only needs `home.blade.php` and `page.blade.php` — every missing page silently falls back to the ecommerce theme.

3. The two required templates

home.blade.php

Rendered at the site root. Receives the CMS "home" page and its sections plus the standard payload. Start from this [resources/views/frontend/themes/](#) · Admin > Theme Settings

```
<!DOCTYPE html>
<html lang="({ str_replace('_', '-', app()->getLocale()) }}" dir="({ \App\Support\Locale::direction() )
```

```

<head>
    @include('partials.head')
    @include('partials.seo-meta')
    <link rel="stylesheet" href="{{ asset('themes/my-theme/style.css') }}">
</head>
<body>
    @php
        $siteName = \App\Models\Setting::get('site_name', config('app.name'));
    @endphp

    <header>
        <a href="{{ url('/') }}">{{ $siteName }}</a>
        <nav>
            @foreach ($menuItems as $item)
                <a href="{{ url($item->url) }}">{{ $item->label }}</a>
            @endforeach
        </nav>
    </header>

    <main>
        @if ($heroImage = \App\Models\Setting::get('home_hero_image'))
            
        @endif

        @foreach ($sections as $section)
            @foreach ($section->localizedCards() as $card)
                <article>
                    @if ($card['title'])<h2>{{ $card['title'] }}</h2>@endif
                    @if ($card['description'])<p>{{ $card['description'] }}</p>@endif
                    @if ($card['image'])@endif
                </article>
            @endforeach
        @endforeach
    </main>

    <livewire:frontend.chat-widget />
    @fluxScripts
    <script src="{{ asset('themes/my-theme/script.js') }}" defer></script>
</body>
</html>

```

4. Standard page template (page.blade.php)

Used for every non-home page (about, contact, faq, and any page you create). Identical payload to home, scoped to the requested page instead of the home page. The template can reuse a shared partial:

```

<!DOCTYPE html>
<html lang="{{ str_replace('_', '-', app()->getLocale()) }}" dir="{{ \App\Support\Locale::direction() }}">
<head>
    @include('partials.head')
    @include('partials.seo-meta')
    <link rel="stylesheet" href="{{ asset('themes/my-theme/style.css') }}">
</head>
<body>
    @include('themes.my-theme.partials.header')

    <main>
        <h1>{{ $page->getTranslation('title', 'en', false) ?: $title }}</h1>
    </main>
</body>
</html>

```

```

        @foreach ($sections as $section)
            @foreach ($section->localizedCards() as $card)
                @include('themes.my-theme.partials.card', ['card' => $card])
            @endforeach
        @endforeach
    </main>

    @include('themes.my-theme.partials.footer')
    @fluxScripts
</body>
</html>

```

Tip: `$page` can be null on view pages that have no paired CMS Page (shop, blog, brand, tag, favorites, cart...), so guard every `$page` read with `$page?->...`

5. Data your templates receive

Every template gets the same **shared payload** from the frontend controller:

Variable	What it holds
<code>\$page</code>	The current CMS Page model, or null on views without one (shop, blog, tags, cart...).
<code>\$sections</code>	Collection of the page's CMS sections (Puck-built content cards). Empty on pages with no paired Page.
<code>\$title</code>	Page SEO title, else the global SEO title, else the site name — used by both the <code><title></code> and the page body.
<code>\$navPages</code>	Standalone published pages in admin sort order — the nav list (portfolio/default style).
<code>\$menuItems</code>	The admin-managed "Frontend" menu items — the nav list (ecommerce style). Each has <code>->label</code> and <code>->url</code> .
<code>\$showVendorLogin</code>	Whether to show a Vendor Login link (false when the vendor portal is disabled).
<code>\$currentSlug</code>	The current route's slug (home, shop, blog, the page slug...) — handy for active nav states.

Per-view extra data

Template	Extra variables
home	None beyond the shared payload; the hero image/tagline come from settings (see §6).
shop	<code>\$products</code> (paginated), <code>\$categories</code> , <code>\$brands</code> , <code>\$tags</code> , <code>\$attributeFacets</code> , <code>\$priceBounds</code> , <code>\$filters</code>
product	<code>\$product</code> (with gallery, brand, tags, categories, faqs, page), <code>\$related</code>
category	<code>\$category</code> , <code>\$children</code> , <code>\$products</code>
brand / tag	<code>\$brand</code> / <code>\$tag</code> , plus <code>\$products</code>
blog	<code>\$posts</code> (paginated, with category & user), <code>\$categories</code>
post	<code>\$post</code> , <code>\$related</code> (other posts)

Template	Extra variables
favorites	\$products (paginated favorite products)
cart / checkout	No extra data — rendering happens in the CartPage/Checkout Livewire components (see §8).
order-confirmation	\$order (with items)
account.*	dashboard/orders: \$orders; order: \$order. Account pages only exist while the ecommerce theme is active.

6. Settings, routes & helpers

Every setting is read with `\App\Models\Setting::get('key', default)`. Settings the frontend commonly uses:

Setting	Purpose
site_name	Site/brand name shown in headers, footers, <code><title></code> .
site_icon / site_icon_white	Logo image URLs (white variant for dark headers).
site_tagline	Short line under the site name on the homepage hero.
home_hero_image	Homepage hero banner image URL.
home_promo_banner_1/2	Two promotional strips below the hero.
contact_email / contact_address	Contact details for the footer / contact section.
chat_widget_enabled	Show a chat widget — the portfolio template gates <code><livewire:frontend.chat-widget /></code> on it.
seo_meta_title / seo_meta_description	Global SEO fallbacks (used by the seo-meta partial).

Link to pages and routes

Use the named routes from your templates: `route('shop')` for the catalog, `route('blog')` for the blog, `route('products.show', ['slug' => $product->slug])`, `route('shop.category', ['slug' => $category->page->slug])`, `route('cart')`, `route('checkout')`, `route('favorites')`, and `url($menuItem->url)` or `url($page->slug)` for CMS pages. Right-click anywhere on the live site — "Copy link" — to grab real URLs while you prototype.

Language & direction

The site is bilingual (English / Bengali). Use `__('...')` for interface strings, `app()->getLocale()` for the current locale, and translatable model fields are JSON arrays — read them with `$page->getTranslation('title', 'en', false)` or `$post->getTranslation('title', app()->getLocale(), false)`. Score the `dir` attribute on `<html>` with `\App\Support\Locale::direction()`.

7. CSS & JavaScript (optional)

Blade files are the only thing a theme strictly needs, but most themes ship a stylesheet. Put static assets in `public/themes/<slug>/` and link them with Laravel's `asset()` helper — the portfolio theme is a working example:

```
public/themes/my-theme/
■■■ style.css
■■■ script.js
```

If you skip your own stylesheet, the site's global CSS (Tailwind, via the `@vite` assets in `partials.head`) is already available — the ecommerce theme builds its whole design on Tailwind utility classes with no theme CSS file at all.

8. Livewire components you can embed

Rich interactive blocks are Livewire components, not plain Blade partials. Drop them into your templates with `@livewire(...)`. Use these in your theme:

`<livewire:frontend.chat-widget />` — the support chat bubble (portfolio home uses it, gated on the setting).

`CartPage / Checkout` — the cart and checkout Livewire components your `cart.blade.php / checkout.blade.php` wrap (see the ecommerce theme's templates for exact usage).

`@fluxScripts` and `@fluxAppearance` — Flux UI's scripts; include the former before `</body>` exactly as the examples do.

The easiest way to build a feature-complete ecommerce theme is to duplicate the ecommerce folder and restyle it — every interaction (add to cart, favorites, checkout) is already wired.

9. Packaging & installing a theme

Step 1 — build the folder

Create a folder named after the theme (lowercase letters, numbers, dashes, underscores — `my-theme`, not `My Theme`). The name becomes the slug. Put your `home.blade.php`, `page.blade.php`, other templates, and a `partials/` folder inside.

Step 2 — zip it

Zip the **folder itself**, so the zip contains exactly one top-level folder inside (e.g. `my-theme/...`). The installer accepts a single root folder; loose templates at the zip root, multiple folders, or a different folder name than the theme slug will be rejected. (Auto-generated junk like `__MACOSX` and dotfiles is ignored.)

Limits enforced on upload

- 50 MB / 2,000 files maximum — keep themes lean (fonts and images should stay in the media library / CDN).
- Path-traversal and absolute paths are rejected ("`..`" escapes), so every extracted file stays inside the theme folder.
- An existing theme of the same slug is never overwritten — rename the folder and re-zip.

Step 3 — upload in the admin

Open **Admin** → **Theme Settings** → **Install Theme**, choose the zip, and install. The new card appears in the picker immediately (a browser reload refreshes the list).

Step 4 — activate

Click the theme's card in the picker, then **Save Theme Settings**. The Live badge moves to the new active theme and the public site starts rendering it. To go back, pick another theme and save again.

Note: anyone with code-level access can also just drop a folder into `resources/views/frontend/themes/` — it shows up in the picker on the next visit. The zip installer exists so non-developers can hand over themes too.

10. Shipping checklist

- Folder name is a clean slug (`my-theme`), and it matches what you tell the developer.
- `home.blade.php` and `page.blade.php` exist — the site renders without them.

- All optional pages you want (shop, product, blog...) are overridden; everything else falls back to ecommerce gracefully.
- Every \$page read is null-safe with \$page?->....
- Translated fields use getTranslation(); interface strings use __('...').
- Settings are read via Setting::get(); assets linked via asset('themes/<slug>/...').
- Zip = single root folder; uploads and activates cleanly in Theme Settings.
- Tested on the live site: homepage, a content page, and any optional pages you ship.

Good luck, and thanks for building a theme for Codeware!